Paper Review • arXiv 2611.11344

Spatial Harness

Spatial Reasoning in Physics Engines

Grounding Spatial Reasoning in Executable World States with Physics Engines

Chentao Cao*, Zhanke Zhou*, Sangni Duan, Bo Han, Hang Li

TMLR Group, Hong Kong Baptist University • ByteDance Seed • First submitted 2026-11-08

Code: github.com/tmlr-group/Spatial-Harness

TL;DR

In short: “write down” the scene, then let the engine “compute”

Spatial Harness: **Perceiver** fuses question constraints with expert visual estimates (detection, depth, facing, pose) into an **explicit JSON world state** run in **MuJoCo**; **Reasoner** writes Python to turn cameras, measure distances, render views, answering from engine-**computed feedback**, not imagination.

52.70% → 71.14%

MindCube end-to-end accuracy

Qwen3.5-27B , +18.44 pp

+7.52 pp

Beats top baseline pySpatial

Same VLM (Qwen3.5-27B)

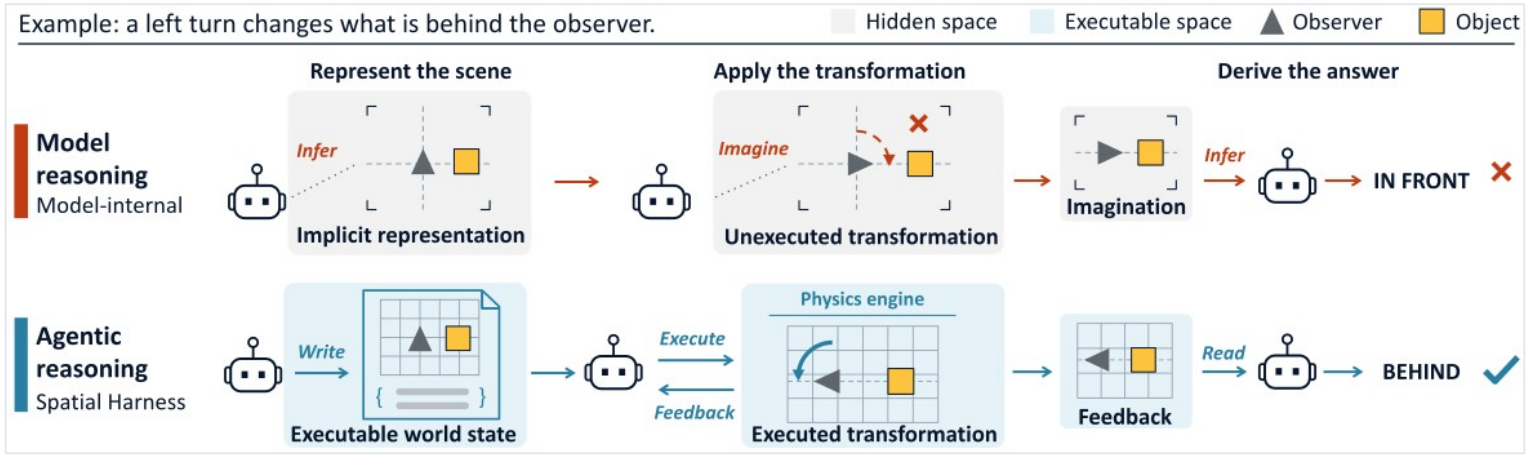
98.25% vs 71.81%

Execute vs. imagine

Pure reasoning (no perception error)

Motivation

Problem: VLMs represent & transform scenes internally; errors go unchecked



Two indistinguishable failures

① Misread layout → consistent error

② Right layout → wrong imagined transform (L/R flip)

Gaps in existing methods

MindJourney's new views lack explicit scene constraints; text maps still leave outcomes to the model

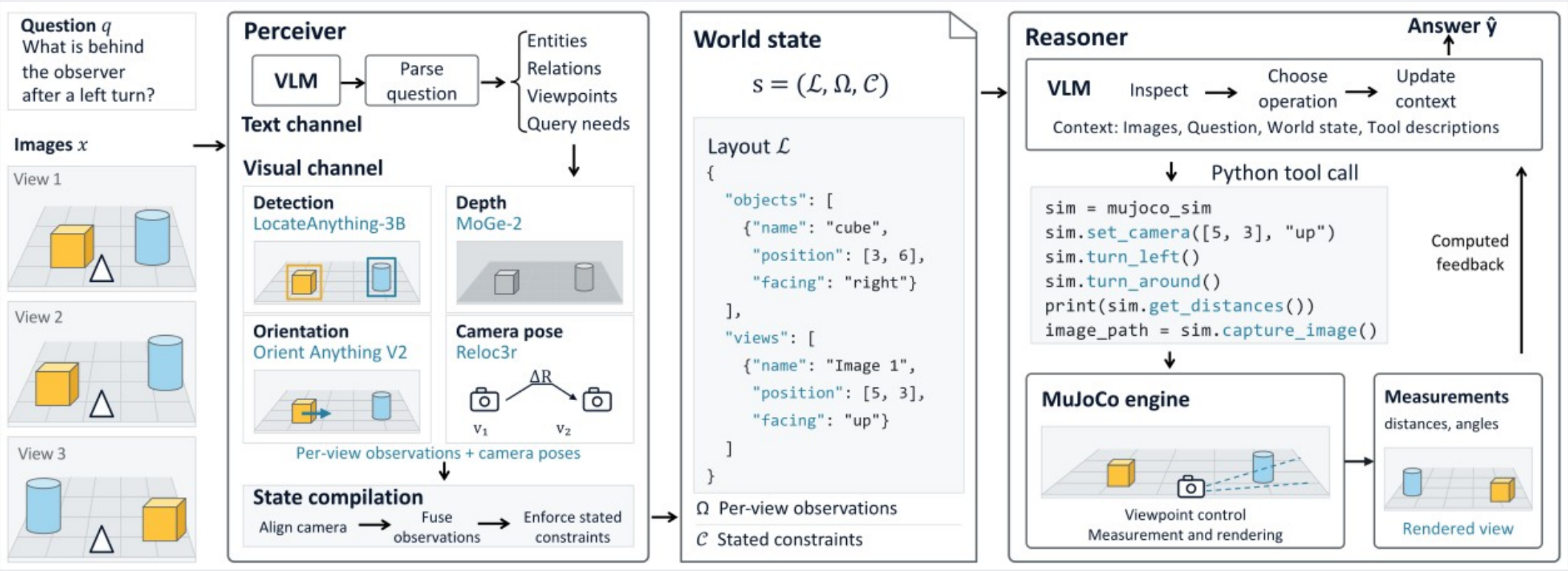
Status quo: GPT-5 on MindCube

Only 47.59%

Random guess: 32.35%

Method

Overview: Perceiver estimates world state, Reasoner runs queries in MuJoCo



Perceiver $\hat{s} = P(x, q)$

Text channel parses q + vision experts estimate geometry → shared-frame layout

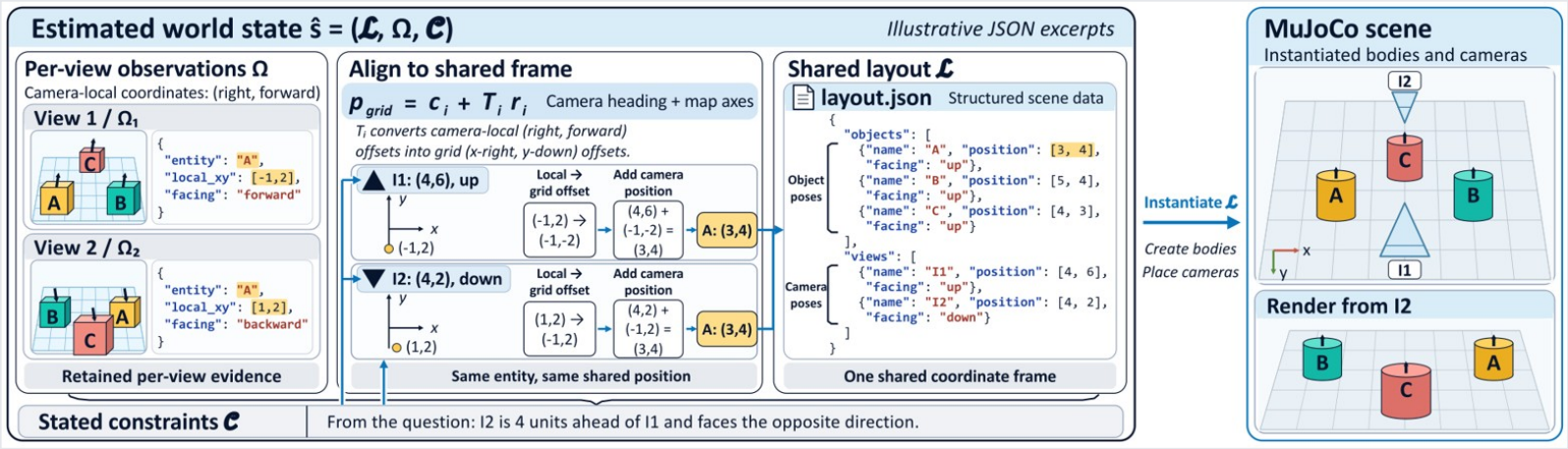
World state $s = (L, \Omega, C)$

Layout L · per-view obs. Ω · constraints C: explicit, checkable, executable

Reasoner $\hat{y} = R(x, q, \hat{s}; E[\hat{s}])$

Calls view/measure/render on engine E[ŝ]; answers from computed feedback

Two-layer state: per-view Ω aligned by camera poses into shared layout L



Ω Observations

Direction, distance and facing of each entity relative to camera i in each image (metric or qualitative)

L Layout

Shared frame with camera 1 as origin; entities (p, θ) and cameras (c, ϕ), used for instantiation

C Constraints

Stated relations/camera motion: strict hard constraints

$$p_{ij} = c_i + R(\phi_i) \cdot r_{ij}$$
$$\theta_{ij} = (\phi_i + \alpha_{ij}) \bmod 2\pi$$

Source: Figure 5 (paper fig04), Eq. 2

Perceiver: language cues + vision experts → layout by reliability priority

Text channel (VLM parses q)

Entities O

Objects & observers in the question

Relations

e.g., “A is left of B”

Viewpoint

e.g. inter-view left turn

Query type

Dir./dist./facing/motion

Vision channel (specialists)

Detection → direction

LocateAnything-3B (+ Qwen3-Reranker for name binding)

Depth → distance

MoGe-2: metric depth & intrinsics

Orientation → facing

Orient Anything V2

Relative camera pose

Reloc3r: rotation & translation direction

Compilation

Camera-heading source priority

Question 0.95 › shared landmark 0.6 › Reloc3r 0.5 › VLM 0.4

Project observations into shared frame

Multiple candidates: highest-confidence position; vote on facing

Grid local search (Eq. 3)

$$\max \Sigma w_k \cdot \mathbb{1}[L \models r_k] - \lambda \Sigma \| p_j - \hat{p}_j \|$$

Question-stated relations weighted 20 (hard); others from image captions/detected bearings

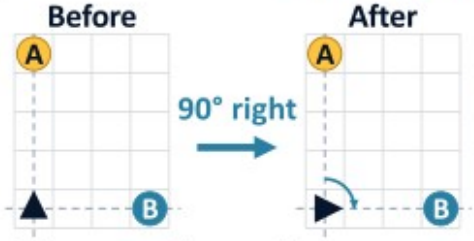
Reasoner: engine as tool—view, measure & render from state

Engine API: mujoco_sim

Example shown after a right turn; distances use state units. *Representative calls shown; complete signatures in the appendix.*

Viewpoint control — Change camera pose

`turn_right()`



Heading changes; position is unchanged

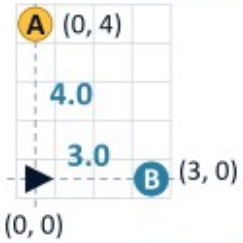
Other control examples

`set_camera(position, facing)`	Place
`rotate_left(angle)`	Rotate
`move_forward(distance)`	Translate

Camera pose persists across calls

Measurement — Read geometric quantities

`get_distances()`




```
{"A": 4.0,
 "B": 3.0}
```

`get_current_view()` Camera position and heading

`Camera: (0, 0), facing right`

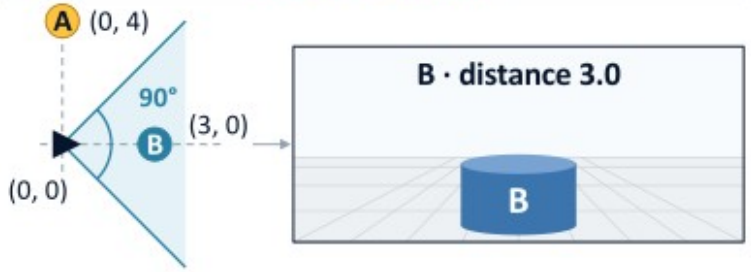
`describe_camera_motion(...)`

Motion between named views

Structured geometric feedback

Rendering — Render the current view

`capture_image()`



Return: image path

Tool feedback: rendered view + object metadata

`Objects in view: B · 3.0`

Visual feedback from the current pose

Interaction loop (Eq. 4)

$$a_t = R(h_t),\ f_t = E[\hat{s}](a_t),\ h_{t+1} = h_t \oplus (a_t, f_t)$$

At most $T_{max} = 5$ rounds; answers accepted only after ≥ 1 engine call; feedback is deterministic and consistent with $\hat{s}$

Proxy representation

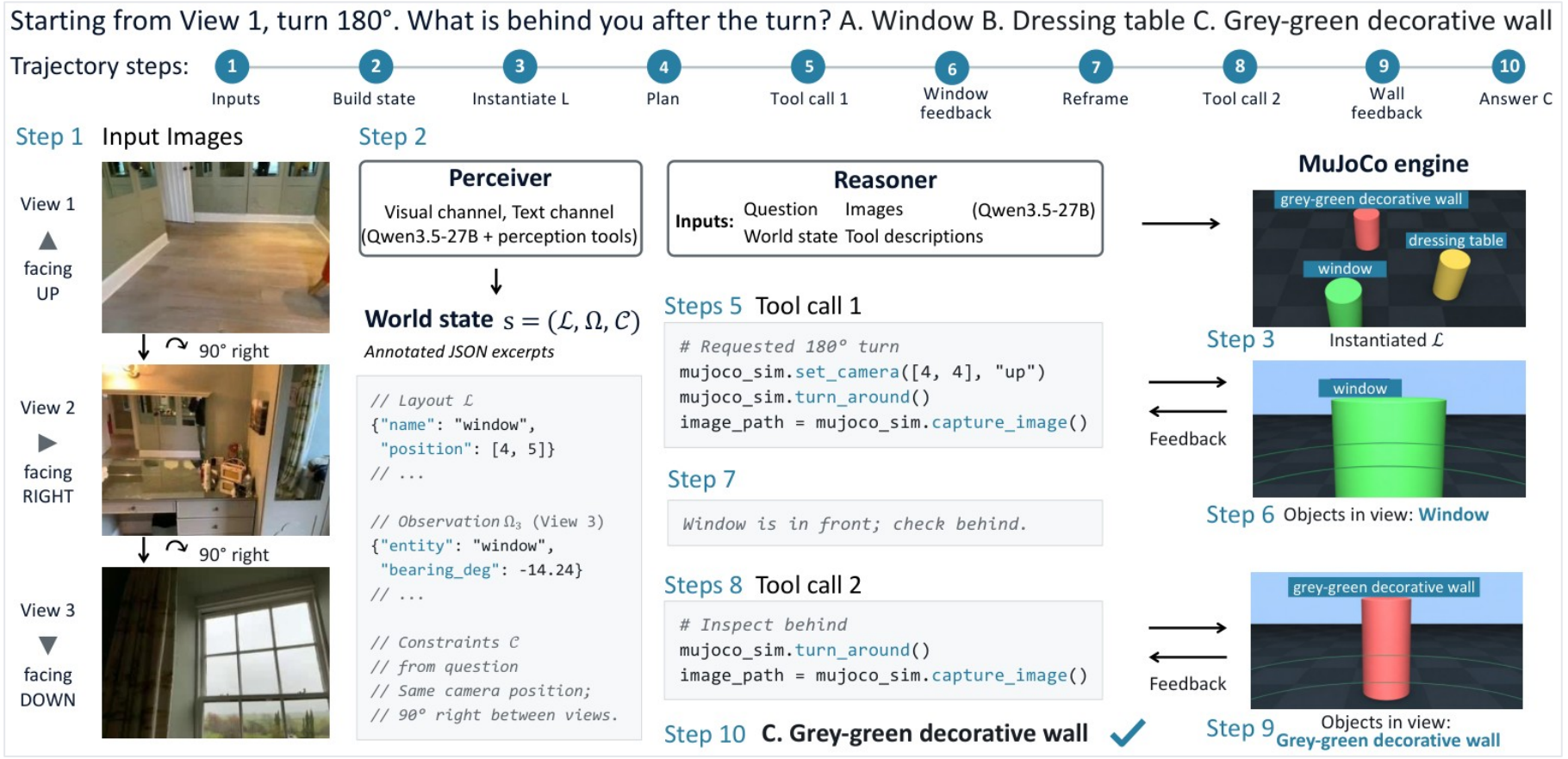
Entity = labeled cylinder with a facing marker; no mesh reconstruction

Routing

Questions the state cannot describe (appearance/occlusion) → fall back to direct VLM answer

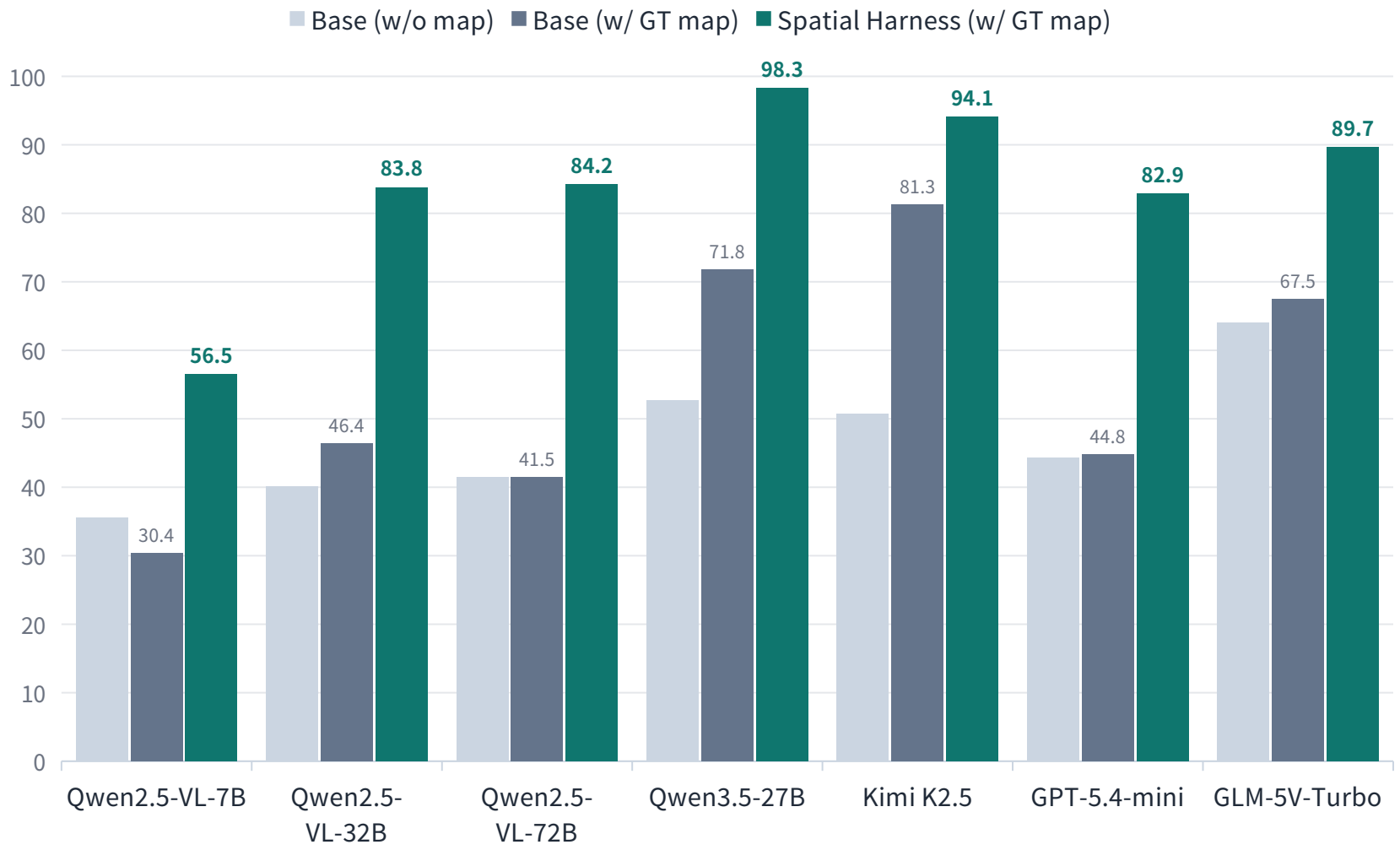
Source: Figure 6 (paper fig05), Eq. 4, Appendix C.3

Example: 180° turn shows “window ahead”; turn again for “behind”



1–3	Build & instantiate the state
4–6	turn_around → sees window
7	Question asks “behind” → replan
8–10	Turn again → gray-green accent wall → pick C

With the same GT map, execution wins on all 7 VLMs: up to 98.25%



Qwen3.5-27B • same map

71.81 → 98.25

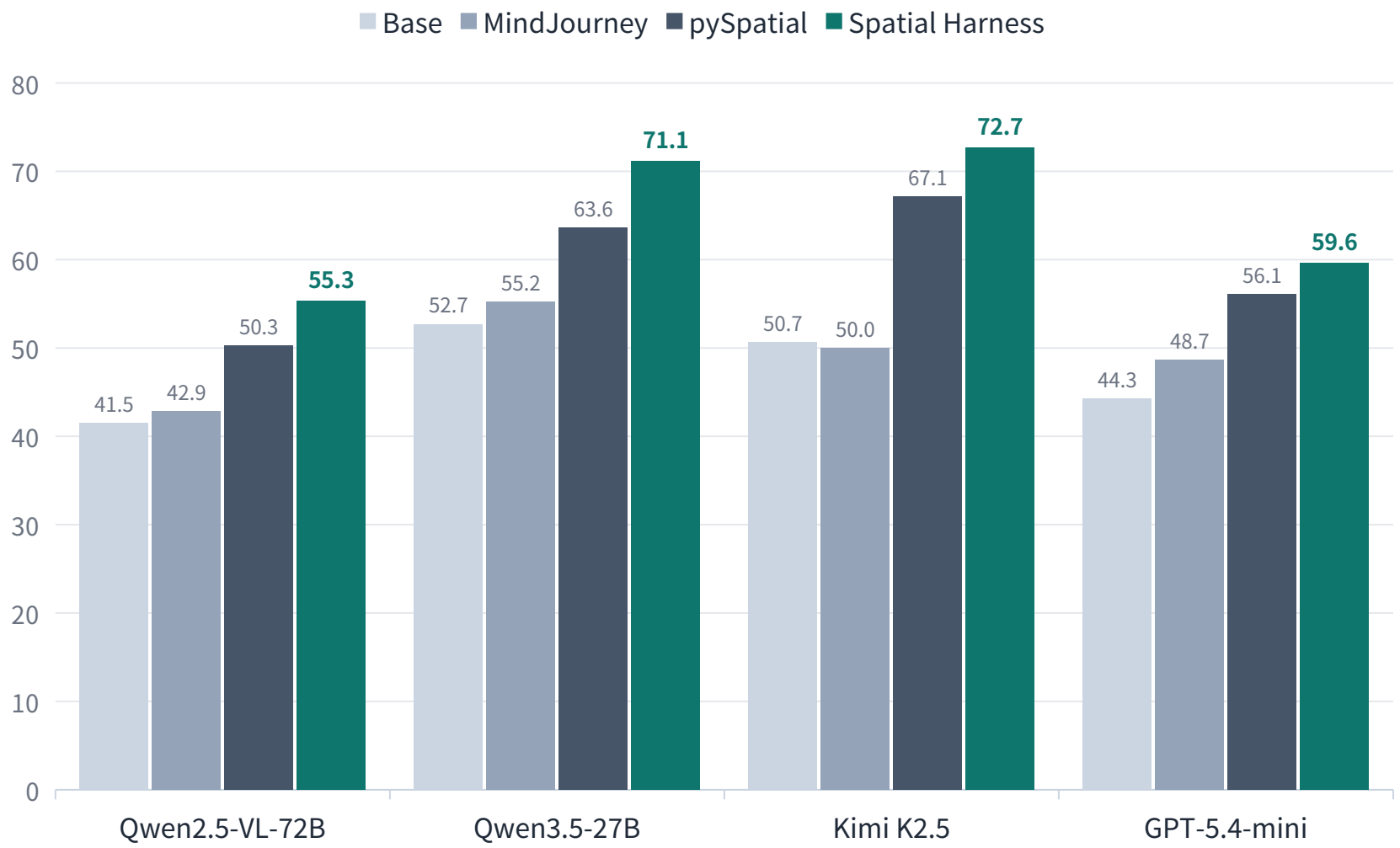
Gains cannot be attributed to better scene estimation

- Text maps help inconsistently: 7B model **35.59 → 30.35**
- Execution vs. Base (with map): **+12.83 to +42.76 pp**
- Std. dev. over 3 runs mostly < 1 pp (Table 11)

Note: Qwen2.5-VL-7B drops on Rotation (35.50 → 34.33); small models use tools poorly

Source: Table 1 / Figure 3 (MindCube, GT-map setting)

End-to-end: first on all 4 VLMs, +5.40 pp over pySpatial on average



MindJourney : ⩽ +4.42 pp

Video world models imagine new views; generated frames lack explicit scene constraints

pySpatial < Base on some

Reconstructs 3D from pixels; cannot absorb question-stated info such as camera motion

Spatial Harness

State absorbs both images and question; all feedback computed from one state

Source: Table 2 (MindCube, end-to-end)

Benchmarks: clear SAT/SpatialRGPT leads; MMSI-Bench backbone-bound

Model	Method	SAT	MMSI-Bench	SpatialRGPT	Avg
Qwen2.5-VL-72B	Base	68.60	29.60	40.72	46.31
	MindJourney	68.60	30.60	39.12	46.11
	pySpatial	55.40	**33.20**	31.64	40.08
	Spatial Harness	**71.60**	29.00	**84.11**	**61.57**
Qwen3.5-27B	Base	81.80	33.40	48.20	54.47
	MindJourney	78.40	33.00	45.66	52.35
	pySpatial	68.00	36.70	39.79	48.16
	Spatial Harness	**83.20**	**39.50**	**83.31**	**68.67**

SAT: single-image; recon. suffers

pySpatial is 13.2 / 13.8 pp below Base; ours leads slightly at 71.60 / 83.20

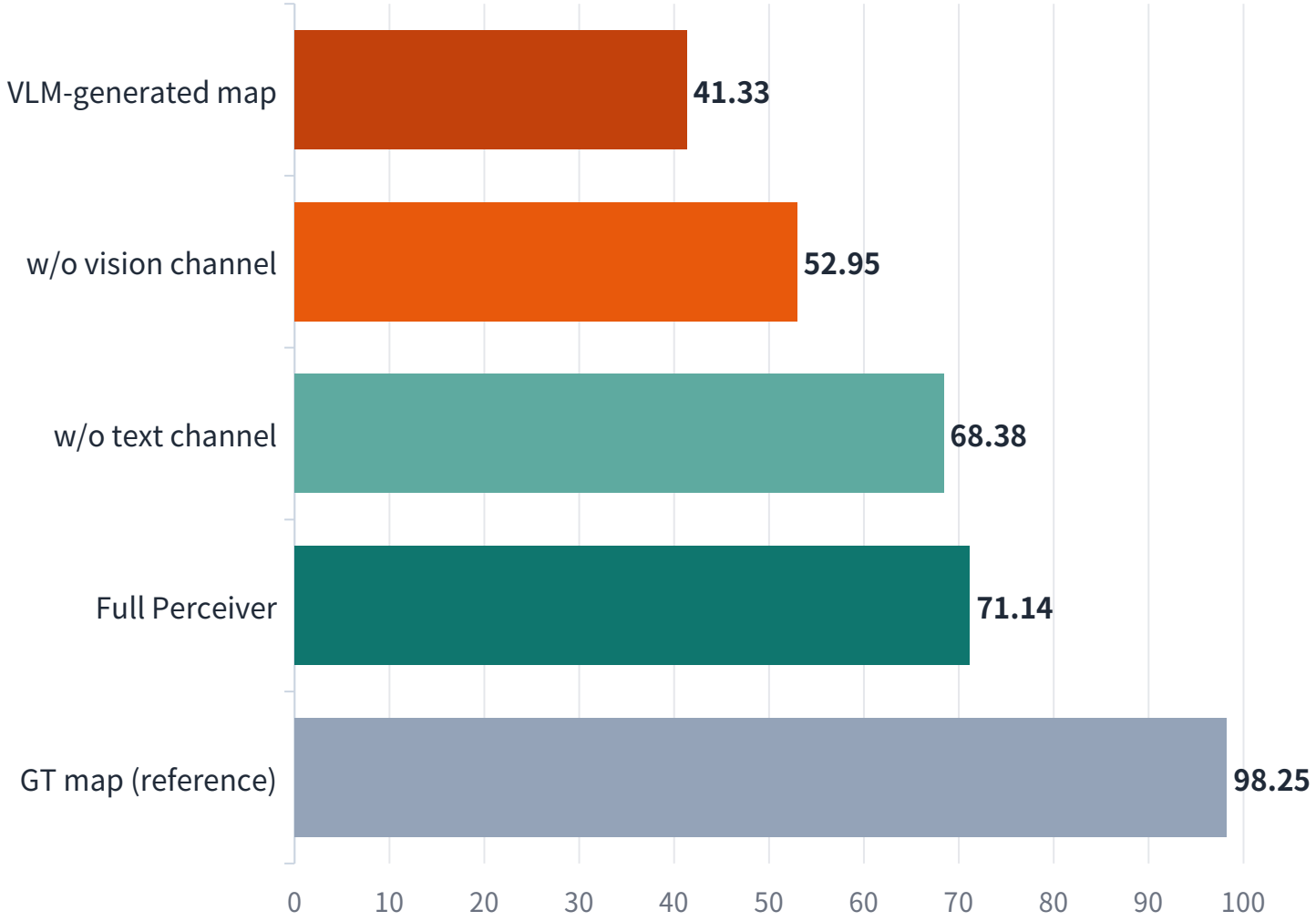
MMSI-Bench: backbone-dependent

27B: 33.40 → 39.50; 72B flat (29.00 vs 29.60), often misreads coordinates, ignores feedback

SpatialRGPT： +35 ~ +43 pp

Distance/size from metric-depth tool and engine measurement; direction questions 98.13%

Ablation ①: state must be structured; vision channel helps most



In-channel modules (Figure 7)

w/o camera/view constraints
Rotation 81.5 → 48.0
62.29

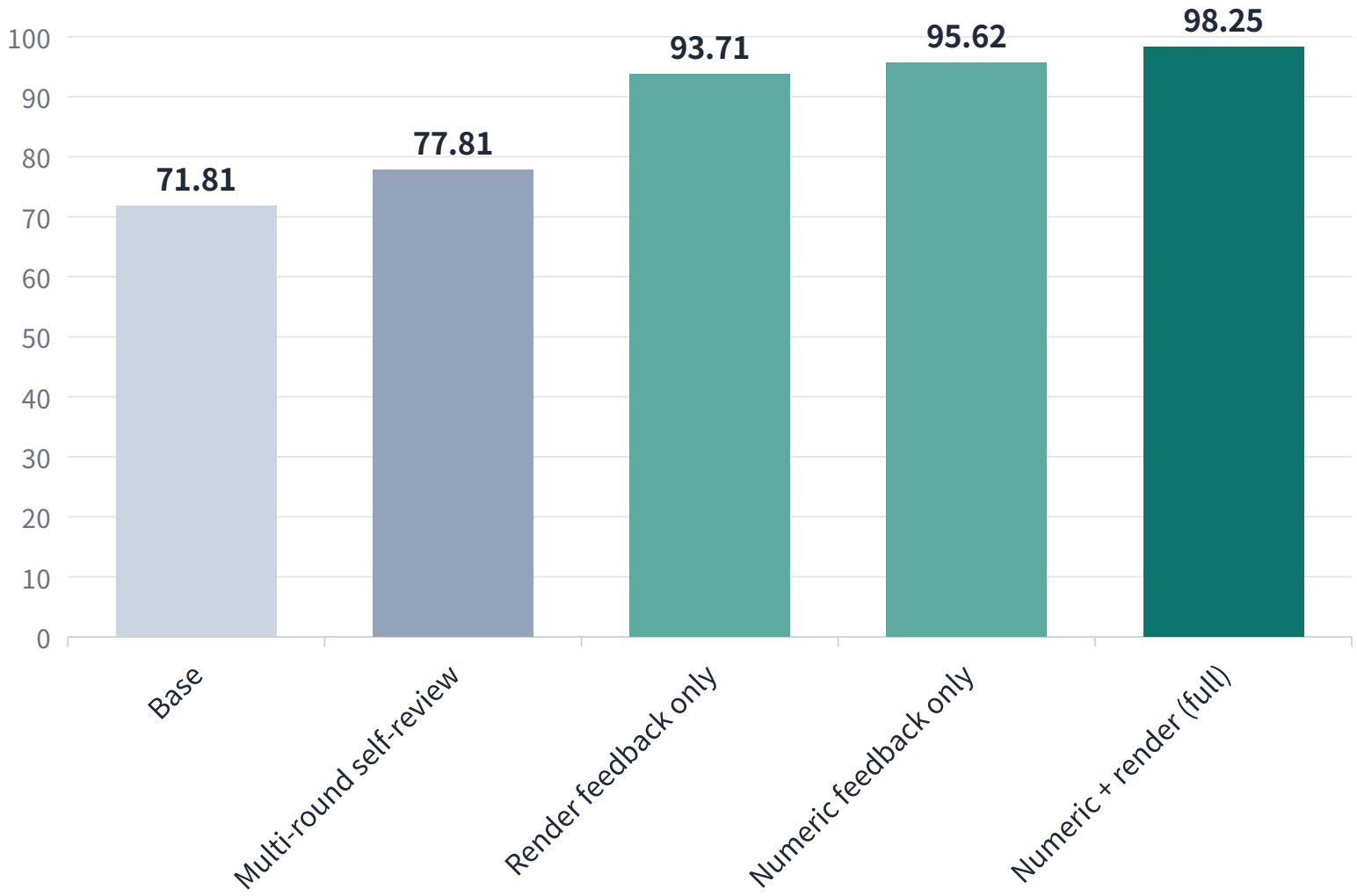
Relative pose → VLM
Largest drop; Among/Around affected
54.10

Facing est. → VLM
Only 36/1050 questions use facing
70.57

Full 71.14 vs. GT 98.25: still a **27 pp** gap → perception is the main bottleneck

Source: Table 4, Figure 7 (MindCube, Qwen3.5-27B)

Ablation ②: feedback > more turns; numeric & render complement



Extra rounds only

+6.00 pp

71.81 → 77.81, no new scene info

Any engine feedback

> 90%

Numeric 95.62 / render 93.71

Combined

98.25%

Measures give precision, renders intuition

Source: Table 5 (MindCube, Qwen3.5-27B, fixed GT map)

State errors: camera heading worst; small position shifts tolerable

Single-factor perturbation (GT map)	Low	Mid	High
One object displaced	1 cell 92.13	2 cells 89.40	3 cells 87.78
One camera's yaw	⩽30° 98.10	**30–90° 84.89**	**90–180° 76.67**
Object facing rotated 90°	1 object 96.06	N/A	2 objects 94.32

Unperturbed 98.25%; facing on 364/129 facing-labeled Qs (ref. 97.53/98.45)

99.52%

Map validity

≈72%

Annotated-entity coverage

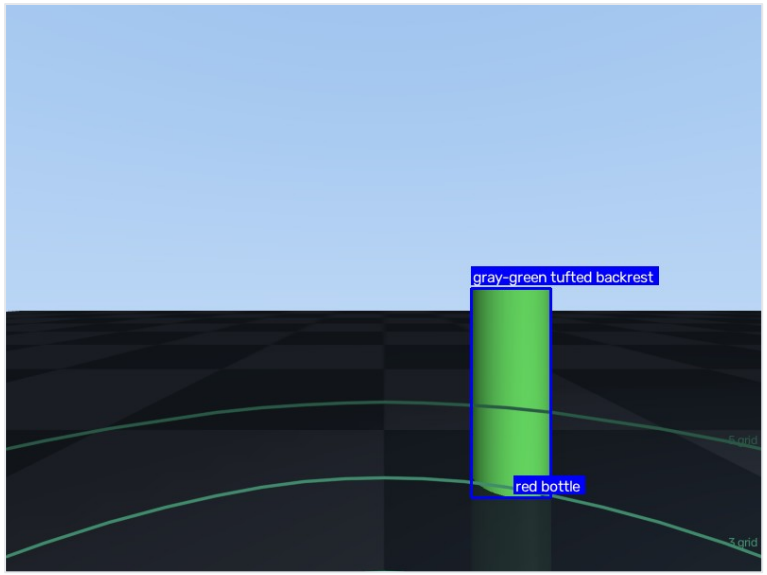
Camera yaw perturbation → accuracy

Estimated state quality of Perceiver (Qwen3.5-27B) on 1,050 questions (Figure 8)

Maps are almost all valid, but **~28% of entities are still missed**; more accurate maps should further improve end-to-end results

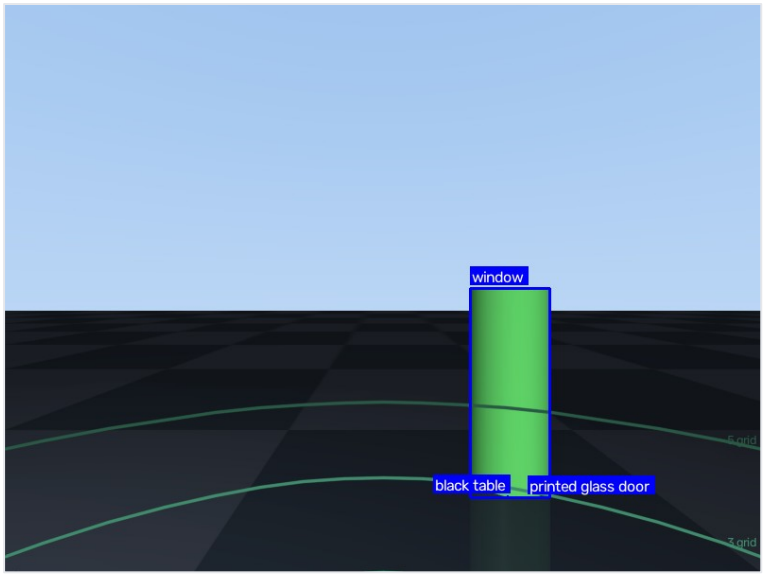
Case study: engine gives correct evidence but can't force adoption

Success • Qwen3.5-27B **Img 4: what's left of the red bottle?**



- set_camera([6,5], "left") → turn_left() → capture_image()
- Only object in view: **gray-green tufted backrest** (3.2) → picks B ✓
- Base took “left” as straight ahead in all 3 runs, picked curtains ✗

Failure • Kimi K2.5 **Img 3: what's left of the toy train?**



- After turn_left() in round 3, the engine renders **window** (correct answer A)
- Model reinterprets it as “the train's own left”, insists on D (wall) ✗
- Same model correct in its other 2 runs; Qwen3.5-27B correct in all 3

Source: Appendix E, Figures 21 & 23 (paper fig08, fig10_3)

Verdict: strong, but engine ≈ geometry calc.; perception lags

Strengths

Diagnostic perception/reasoning decoupling: same GT map, 71.81 vs 98.25, pinpoints flawed "imagined transforms"

Language constraints enter the state: question-stated camera motion as hard constraint; removing it drops Rotation 81.5 → 48.0

Training-free, model-agnostic: effective on 7 open/closed VLMs; temperature 0, 3-run variance and all prompts reported

Solid feedback ablation: self-review gives only +6 pp, ruling out the "more rounds" confound

Limitations & Questions

Engine does geometry only: static scenes, cylinder proxies, no dynamics; any 3D renderer would likely be equivalent

Perception bottleneck: 27 pp end-to-end gap to GT; entity coverage ~72%

Tailored to MindCube: 10×10 grid, 4 facing directions, many hand-set weights; no gain for 72B on MMSI

Source of gains: SpatialRGPT gains come mainly from MoGe-2 metric depth

Not reported: inference cost/latency and routing-fallback rate

Takeaways

VLMs understand and orchestrate; engines compute

01

Explicit + executable

World state s=(L,Ω,C) makes perception checkable and spatial transforms computable

02

Reasoning bottleneck quantified

Same GT map: execution 98.25% vs imagination 71.81%

03

Best end-to-end across the board

MindCube: first on all 4 VLMs; Qwen3.5-27B 52.70 → 71.14

04

Next steps

Improve perception coverage and camera poses; add a time dimension for video/motion questions

arXiv 2611.11344 • Code: github.com/tmlr-group/Spatial-Harness